

DATABASE SYSTEMS BOOTCAMP

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

Complete every task using DB Fiddle (or any MySQL environment of your choice). Read each instruction carefully before writing your query — some tasks build on the schema you create in Part 1.

Before you start: you only need a browser and DB Fiddle (db-fiddle.com, set to MySQL). No installation or account is required.

Part 1 — Build Your Own Practice Schema

Before attempting the tasks below, design and create your own small database. Do not copy a schema from anywhere — write your own CREATE TABLE statements based on the description below, then insert your own sample data using INSERT statements.

Table	Columns	Notes
customers	id (primary key), name, email (unique), signup_date	Add at least 15–20 sample customers.
restaurants	id (primary key), name, cuisine	Add at least 8–10 sample restaurants.
orders	id (primary key), customer_id, restaurant_id, total, status, created_at	Add at least 40–50 sample orders. customer_id and restaurant_id must match real ids from your other two tables.
accounts	id (primary key), name, balance	Just 2 rows — e.g. one account named Alice and one named Bob — used only for Task 1.

Customers :

	id [PK] integer	name character varying (100)	email character varying (100)	signup_date date
1	1	Haruto Sato	haruto.sato@gmail.com	2026-01-05
2	2	Yui Tanaka	yui.tanaka@gmail.com	2026-01-08
3	3	Ren Nakamura	ren.nakamura@gmail.com	2026-01-12
4	4	Aoi Suzuki	aoi.suzuki@gmail.com	2026-01-15
5	5	Kaito Yamamoto	kaito.yamamoto@gmail.c...	2026-01-20
6	6	Lukas Müller	lukas.mueller@gmail.com	2026-01-25
7	7	Emma Schneider	emma.schneider@gmail.c...	2026-02-01
8	8	Felix Weber	felix.weber@gmail.com	2026-02-04
9	9	Mia Fischer	mia.fischer@gmail.com	2026-02-10
10	10	Jonas Wagner	jonas.wagner@gmail.com	2026-02-15
11	11	Lucas Silva	lucas.silva@gmail.com	2026-02-20
12	12	Sofia Oliveira	sofia.oliveira@gmail.com	2026-02-25
13	13	Miguel Santos	miguel.santos@gmail.com	2026-03-01
14	14	Helena Costa	helena.costa@gmail.com	2026-03-05
15	15	Arthur Lima	arthur.lima@gmail.com	2026-03-10
16	16	Omar Hassan	omar.hassan@gmail.com	2026-03-15
17	17	Nour Ali	nour.ali@gmail.com	2026-03-20
18	18	Youssef Ibrahim	youssef.ibrahim@gmail.c...	2026-03-25
19	19	Mariam Mahmoud	mariam.mahmoud@gmail...	2026-03-28
20	20	Karim Mostafa	karim.mostafa@gmail.com	2026-04-01

Restaurants :

	id [PK] integer	name character varying (150)	cuisine character varying (100)
1	1	Sakura Kitchen	Japanese
2	2	Tokyo Bites	Japanese
3	3	Samurai Grill	Japanese
4	4	Berlin Bistro	German
5	5	Code Cafe	Fusion
6	6	Amazon Spice	Brazilian
7	7	Rio Flavors	Brazilian
8	8	Carnival Kitchen	Brazilian
9	9	Nile Taste	Egyptian
10	10	Desert Delight	Middle Eastern

Orders :

	id [PK] integer	customer_id integer	restaurant_id integer	total numeric (8,2)	status character varying (20)	created_at timestamp without time zone
1	1	1	1	499.00	Completed	2026-04-05 10:15:00
2	2	1	3	599.00	Completed	2026-04-18 14:30:00
3	3	2	2	349.50	Pending	2026-04-20 09:45:00
4	4	3	1	499.00	Completed	2026-05-02 16:20:00
5	5	4	2	349.50	Cancelled	2026-05-05 11:10:00
6	6	5	3	599.00	Completed	2026-05-09 13:50:00
7	7	6	4	420.00	Completed	2026-05-15 18:25:00
8	8	7	5	799.00	Pending	2026-05-18 20:10:00
9	9	8	4	420.00	Completed	2026-05-22 08:40:00
10	10	9	5	799.00	Completed	2026-05-28 15:35:00
11	11	10	5	799.00	Completed	2026-06-01 09:10:00
12	12	11	6	550.00	Completed	2026-06-03 13:25:00
13	13	11	7	380.00	Pending	2026-06-07 15:00:00
14	14	12	8	610.00	Completed	2026-06-10 11:30:00
15	15	12	6	550.00	Completed	2026-06-12 17:20:00
16	16	13	7	380.00	Cancelled	2026-06-14 10:40:00
17	17	14	8	610.00	Completed	2026-06-18 18:50:00
18	18	15	6	550.00	Completed	2026-06-22 12:45:00
19	19	16	9	700.00	Pending	2026-06-25 09:15:00
20	20	17	10	450.00	Completed	2026-06-28 16:35:00
21	21	17	9	700.00	Completed	2026-07-01 10:10:00
22	22	18	10	450.00	Completed	2026-07-03 11:45:00
23	23	18	9	700.00	Pending	2026-07-05 14:20:00
24	24	19	10	450.00	Completed	2026-07-07 16:30:00
25	25	1	2	349.50	Completed	2026-07-10 09:15:00
26	26	3	3	599.00	Completed	2026-07-12 18:10:00
27	27	5	1	499.00	Pending	2026-07-15 12:30:00
28	28	6	5	799.00	Completed	2026-07-18 15:25:00
29	29	8	4	420.00	Cancelled	2026-07-20 17:40:00
30	30	10	6	550.00	Completed	2026-07-22 12:50:00
31	31	11	8	610.00	Completed	2026-07-24 10:35:00
32	32	12	7	380.00	Completed	2026-07-26 11:15:00
33	33	13	6	550.00	Pending	2026-07-28 14:45:00
34	34	14	9	700.00	Completed	2026-07-30 09:20:00
35	35	15	10	450.00	Cancelled	2026-08-02 18:00:00
36	36	16	8	610.00	Completed	2026-08-04 13:10:00
37	37	17	7	380.00	Completed	2026-08-06 16:25:00
38	38	18	6	550.00	Completed	2026-08-08 12:40:00
39	39	19	9	700.00	Pending	2026-08-10 15:30:00
41	41	3	5	799.00	Completed	2026-08-15 09:30:00
42	42	5	2	349.50	Completed	2026-08-18 10:45:00
43	43	6	1	499.00	Pending	2026-08-20 13:20:00
44	44	7	3	599.00	Completed	2026-08-22 16:10:00
45	45	9	4	420.00	Completed	2026-08-24 11:00:00
46	46	10	8	610.00	Cancelled	2026-08-26 17:30:00
47	47	11	9	700.00	Completed	2026-08-28 14:40:00
48	48	14	10	450.00	Completed	2026-08-30 12:15:00
49	49	16	7	380.00	Completed	2026-09-02 18:20:00
50	50	19	6	550.00	Pending	2026-09-05 15:50:00

Accounts :

	id [PK] integer	name character varying (100)	balance numeric (10,2)
1	1	Global Savings	15000.00
2	2	Business Wallet	8000.00

Part 2 — Tasks

TASK 1 Transactions (ACID)

Using your accounts table:

- a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

```

1  UPDATE accounts
2  SET balance = balance - 200
3  WHERE id = 1;
4  UPDATE accounts
5  SET balance = balance + 200
6  WHERE id = 2;
7
8  SELECT * FROM accounts;
9
10 ROLLBACK;
11
12 SELECT * FROM accounts;

```

id	name	balance
1	Global Savings	15000.00
2	Business Wallet	8000.00

- b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

```

1  UPDATE accounts
2  SET balance = balance - 200
3  WHERE id = 1;
4  UPDATE accounts
5  SET balance = balance + 200
6  WHERE id = 2;
7
8  COMMIT;
9
10 SELECT * FROM accounts;

```

id	name	balance
1	Global Savings	14800.00
2	Business Wallet	8200.00

- c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.
- ⇒ The ACID property that guarantees money is never lost during a transaction is *Atomicity*. Atomicity ensures that all operations in a transaction are completed successfully or none of them are applied. If any error occurs during the transfer, PostgreSQL rolls back the entire transaction, keeping the account balances consistent.

TASK 2 Joins

Using your customers and orders tables:

- a) Write an INNER JOIN that lists every customer who has placed at least one order.

Query Query History			
1	SELECT		
2	c.id,		
3	c.name,		
4	c.email,		
5	o.id AS order_id		
6	FROM customers c		
7	INNER JOIN orders o		
8	ON c.id = o.customer_id;		

Data Output Messages Notifications			
		SQL	Showing rows: 1 to 50 Page No: 1 of 1
id	name	email	order_id
integer	character varying (100)	character varying (100)	integer
1	Haruto Sato	haruto.sato@gmail.com	1
2	Haruto Sato	haruto.sato@gmail.com	2
3	Yui Tanaka	yui.tanaka@gmail.com	3
4	Ren Nakamura	ren.nakamura@gmail.com	4
5	Aoi Suzuki	aoi.suzuki@gmail.com	5
6	Kaito Yamamoto	kaito.yamamoto@gmail.c...	6
7	Lukas Müller	lukas.mueller@gmail.com	7
8	Emma Schneider	emma.schneider@gmail.c...	8
9	Felix Weber	felix.weber@gmail.com	9
10	Mia Fischer	mia.fischer@gmail.com	10
11	Jonas Wagner	jonas.wagner@gmail.com	11
12	Lucas Silva	lucas.silva@gmail.com	12
13	Lucas Silva	lucas.silva@gmail.com	13
14	Sofia Oliveira	sofia.oliveira@gmail.com	14
15	Sofia Oliveira	sofia.oliveira@gmail.com	15
16	Miguel Santos	miguel.santos@gmail.com	16
17	Helena Costa	helena.costa@gmail.com	17
18	Arthur Lima	arthur.lima@gmail.com	18
19	Omar Hassan	omar.hassan@gmail.com	19
20	Nour Ali	nour.ali@gmail.com	20
21	Nour Ali	nour.ali@gmail.com	21
22	Youssef Ibrahim	youssef.ibrahim@gmail.c...	22
23	Youssef Ibrahim	youssef.ibrahim@gmail.c...	23
24	Mariam Mahmoud	mariam.mahmoud@gmail...	24
25	Haruto Sato	haruto.sato@gmail.com	25
26	Ren Nakamura	ren.nakamura@gmail.com	26
27	Kaito Yamamoto	kaito.yamamoto@gmail.c...	27
28	Lukas Müller	lukas.mueller@gmail.com	28
29	Felix Weber	felix.weber@gmail.com	29
30	Jonas Wagner	jonas.wagner@gmail.com	30
31	Lucas Silva	lucas.silva@gmail.com	31
32	Sofia Oliveira	sofia.oliveira@gmail.com	32
33	Miguel Santos	miguel.santos@gmail.com	33
34	Helena Costa	helena.costa@gmail.com	34
35	Arthur Lima	arthur.lima@gmail.com	35
36	Omar Hassan	omar.hassan@gmail.com	36
37	Nour Ali	nour.ali@gmail.com	37
38	Youssef Ibrahim	youssef.ibrahim@gmail.c...	38
39	Mariam Mahmoud	mariam.mahmoud@gmail...	39
40	Haruto Sato	haruto.sato@gmail.com	40
41	Ren Nakamura	ren.nakamura@gmail.com	41
42	Kaito Yamamoto	kaito.yamamoto@gmail.c...	42
43	Lukas Müller	lukas.mueller@gmail.com	43
44	Emma Schneider	emma.schneider@gmail.c...	44
45	Mia Fischer	mia.fischer@gmail.com	45
46	Jonas Wagner	jonas.wagner@gmail.com	46
47	Lucas Silva	lucas.silva@gmail.com	47
48	Helena Costa	helena.costa@gmail.com	48
49	Omar Hassan	omar.hassan@gmail.com	49
50	Mariam Mahmoud	mariam.mahmoud@gmail...	50

Total rows: 50 Query complete 00:00:00.071 CRLF Ln 8, Col 25

- b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

Query

Query History

1

SELECT

2

customers.id,

3

customers.name,

4

customers.email,

5

orders.id AS order_id

6

FROM customers

7

LEFT JOIN orders

8

ON customers.id = orders.customer_id

9

WHERE orders.id IS NULL;

Data Output

Messages

Notifications

- c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).
 ⇒ The marketing team prefers the result of the LEFT JOIN query because it identifies customers who have never placed an order, allowing them to target those customers with promotional offers and encourage their first purchase.

TASK 3 Indexing

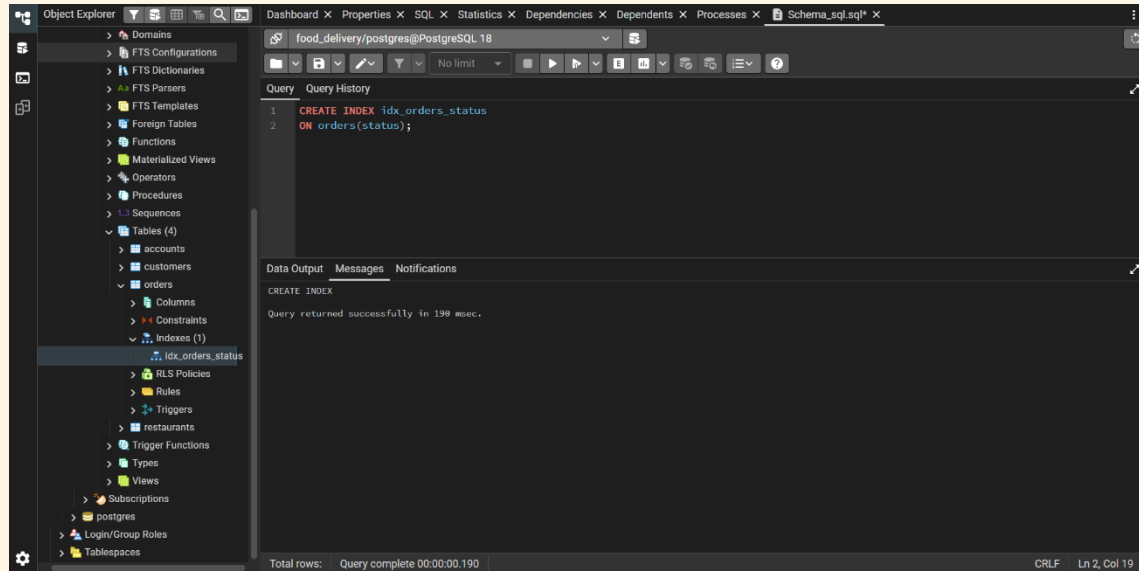
Using your orders table:

- a) Pick a column that is not currently indexed (for example status or created_at).
status
- b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

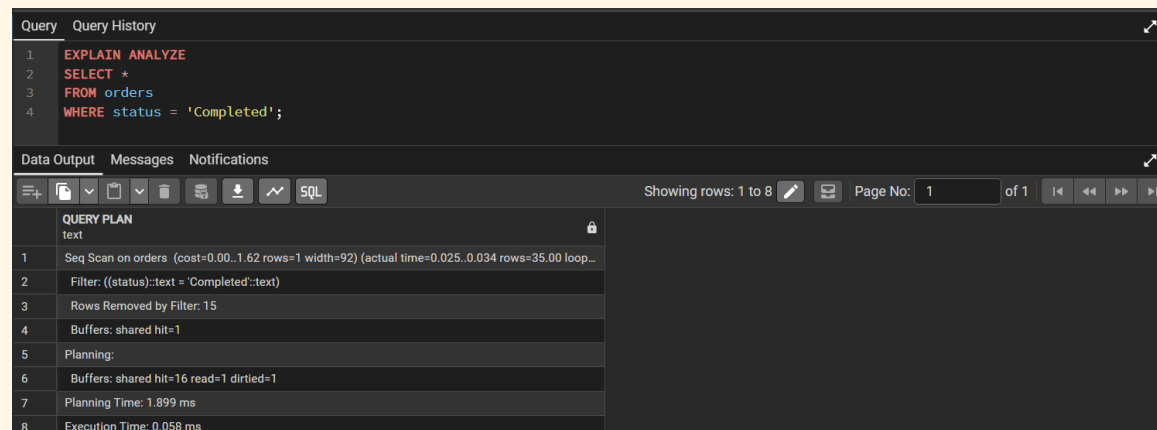
Query		Query History
1	EXPLAIN ANALYZE	
2	SELECT *	
3	FROM orders	
4	WHERE status = 'Completed';	

Data Output		Messages	Notifications
Showing rows: 1 to 8	Page No: 1	of 1	
QUERY PLAN			
1	Seq Scan on orders (cost=0.00..18.50 rows=3 width=92) (actual time=0.040..0.047 rows=35.00 loop=1)		
2	Filter: ((status)::text = 'Completed')::text		
3	Rows Removed by Filter: 15		
4	Buffers: shared hit=1		
5	Planning:		
6	Buffers: shared hit=30		
7	Planning Time: 2.498 ms		
8	Execution Time: 0.083 ms		

c) Create an index on that column.



d) Re-run the exact same query and record the new numbers.



e) Submit a one-line comparison of before vs. after.

⇒ After creating the index, PostgreSQL used the index to search the status column instead of scanning the entire table. Since this dataset contains only 50 records, the performance improvement is small, but the benefit becomes much larger with large datasets.

TASK 4 Query Optimization

Start with this query, rewritten to use your own table and column names:

```
SELECT * FROM orders WHERE customer_id IN (
  SELECT id FROM customers WHERE signup_date >= 'some date'
);
```

a) Rewrite it as a JOIN instead of a subquery.

Query

Query History

1

SELECT

2

orders.*

3

FROM orders

4

INNER JOIN customers

5

ON orders.customer_id = customers.id

6

WHERE customers.signup_date >= '2026-02-01';

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 35

Page No: 1

of 1

	id [PK] integer	customer_id integer	restaurant_id integer	total numeric (8,2)	status character varying (20)	created_at timestamp without time zone
1	8	7	5	799.00	Pending	2026-05-18 20:10:00
2	9	8	4	420.00	Completed	2026-05-22 08:40:00
3	10	9	5	799.00	Completed	2026-05-28 15:35:00
4	11	10	5	799.00	Completed	2026-06-01 09:10:00
5	12	11	6	550.00	Completed	2026-06-03 13:25:00
6	13	11	7	380.00	Pending	2026-06-07 15:00:00
7	14	12	8	610.00	Completed	2026-06-10 11:30:00
8	15	12	6	550.00	Completed	2026-06-12 17:20:00
9	16	13	7	380.00	Cancelled	2026-06-14 10:40:00
10	17	14	8	610.00	Completed	2026-06-18 18:50:00
11	18	15	6	550.00	Completed	2026-06-22 12:45:00
12	19	16	9	700.00	Pending	2026-06-25 09:15:00
13	20	17	10	450.00	Completed	2026-06-28 16:35:00
14	21	17	9	700.00	Completed	2026-07-01 10:10:00
15	22	18	10	450.00	Completed	2026-07-03 11:45:00
16	23	18	9	700.00	Pending	2026-07-05 14:20:00
17	24	19	10	450.00	Completed	2026-07-07 16:30:00
18	29	8	4	420.00	Cancelled	2026-07-20 17:40:00
19	30	10	6	550.00	Completed	2026-07-22 13:50:00
20	31	11	8	610.00	Completed	2026-07-24 10:35:00
21	32	12	7	380.00	Completed	2026-07-26 11:15:00
22	33	13	6	550.00	Pending	2026-07-28 14:45:00
23	34	14	9	700.00	Completed	2026-07-30 09:20:00
24	35	15	10	450.00	Cancelled	2026-08-02 18:00:00
25	36	16	8	610.00	Completed	2026-08-04 13:10:00
26	37	17	7	380.00	Completed	2026-08-06 16:25:00
27	38	18	6	550.00	Completed	2026-08-08 12:40:00
28	39	19	9	700.00	Pending	2026-08-10 15:30:00
29	44	7	3	599.00	Completed	2026-08-22 16:10:00
30	45	9	4	420.00	Completed	2026-08-24 11:00:00
31	46	10	8	610.00	Cancelled	2026-08-26 17:30:00
32	47	11	9	700.00	Completed	2026-08-28 14:40:00
33	48	14	10	450.00	Completed	2026-08-30 12:15:00
34	49	16	7	380.00	Completed	2026-09-02 18:20:00
35	50	19	6	550.00	Pending	2026-09-05 15:50:00

Total rows: 35

Query complete 00:00:00.161

CRLF

Ln 6, Col 45

b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

Query Query History

```

1 SELECT
2   orders.id,
3   customers.name,
4   orders.total,
5   orders.status,
6   orders.created_at
7 FROM orders
8 INNER JOIN customers
9   ON orders.customer_id = customers.id
10 WHERE customers.signup_date >= '2026-02-01';

```

Data Output Messages Notifications

Showing rows: 1 to 35 Page No: 1 of 1

	id	name	total	status	created_at
	integer	character varying (100)	numeric (8,2)	character varying (20)	timestamp without time zone
1	8	Emma Schneider	799.00	Pending	2026-05-18 20:10:00
2	9	Felix Weber	420.00	Completed	2026-05-22 08:40:00
3	10	Mia Fischer	799.00	Completed	2026-05-28 15:35:00
4	11	Jonas Wagner	799.00	Completed	2026-06-01 09:10:00
5	12	Lucas Silva	550.00	Completed	2026-06-03 13:25:00
6	13	Lucas Silva	380.00	Pending	2026-06-07 15:00:00
7	14	Sofia Oliveira	610.00	Completed	2026-06-10 11:30:00
8	15	Sofia Oliveira	550.00	Completed	2026-06-12 17:20:00
9	16	Miguel Santos	380.00	Cancelled	2026-06-14 10:40:00
10	17	Helena Costa	610.00	Completed	2026-06-18 18:50:00
11	18	Arthur Lima	550.00	Completed	2026-06-22 12:45:00
12	19	Omar Hassan	700.00	Pending	2026-06-25 09:15:00
13	20	Nour Ali	450.00	Completed	2026-06-28 16:35:00
14	21	Nour Ali	700.00	Completed	2026-07-01 10:10:00
15	22	Youssef Ibrahim	450.00	Completed	2026-07-03 11:45:00
16	23	Youssef Ibrahim	700.00	Pending	2026-07-05 14:20:00
17	24	Mariam Mahmoud	450.00	Completed	2026-07-07 16:30:00
18	29	Felix Weber	420.00	Cancelled	2026-07-20 17:40:00
19	30	Jonas Wagner	550.00	Completed	2026-07-22 13:50:00
20	31	Lucas Silva	610.00	Completed	2026-07-24 10:35:00
21	32	Sofia Oliveira	380.00	Completed	2026-07-26 11:15:00
22	33	Miguel Santos	550.00	Pending	2026-07-28 14:45:00
23	34	Helena Costa	700.00	Completed	2026-07-30 09:20:00
24	35	Arthur Lima	450.00	Cancelled	2026-08-02 18:00:00
25	36	Omar Hassan	610.00	Completed	2026-08-04 13:10:00
26	37	Nour Ali	380.00	Completed	2026-08-06 16:25:00
27	38	Youssef Ibrahim	550.00	Completed	2026-08-08 12:40:00
28	39	Mariam Mahmoud	700.00	Pending	2026-08-10 15:30:00
29	44	Emma Schneider	599.00	Completed	2026-08-22 16:10:00
30	45	Mia Fischer	420.00	Completed	2026-08-24 11:00:00
31	46	Jonas Wagner	610.00	Cancelled	2026-08-26 17:30:00
32	47	Lucas Silva	700.00	Completed	2026-08-28 14:40:00
33	48	Helena Costa	450.00	Completed	2026-08-30 12:15:00
34	49	Omar Hassan	380.00	Completed	2026-09-02 18:20:00
35	50	Mariam Mahmoud	550.00	Pending	2026-09-05 15:50:00

Total rows: 35 Query complete 00:00:00.146 CRLF Ln 10, Col 45

c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

Original query :

Query Query History

```

1 EXPLAIN ANALYZE
2 SELECT *
3 FROM orders
4 WHERE customer_id IN (
5   SELECT id
6   FROM customers
7   WHERE signup_date >= '2026-02-01'
8 );

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN
Hash
1 Hash Join (cost=12.64..14.47 rows=17 width=92) (actual time=0.081..0.107 rows=33.00 loops=1)
2 Hash Cond: (orders.customer_id = customers.id)
3 Buffers: shared hit=2
4 -> Seq Scan on orders (cost=0.00..1.50 rows=50 width=92) (actual time=0.029..0.034 rows=50.00 loops=1)
5 Buffers: shared hit=1
6 -> Hash (cost=12.12..12.12 rows=57 width=4) (actual time=0.036..0.037 rows=14.00 loops=1)
7 Buckets: 1024 Batches: 1 Memory Usage: 968
8 Buffers: shared hit=1
9 -> Seq Scan on customers (cost=0.00..12.12 rows=57 width=4) (actual time=0.022..0.026 rows=14.00 loops=1)
10 Filter: (signup_date >= '2026-02-01'::date)
11 Rows Removed by Filter: 6
12 Buffers: shared hit=1
13 Planning Time: 0.233 ms
14 Execution Time: 0.160 ms

Total rows: 14 Query complete 00:00:00.077 CRLF Ln 8, Col 3

Final rewritten version :

The screenshot shows a database query editor with a dark theme. The top panel displays a SQL query with line numbers 1 through 6. The query is an `EXPLAIN ANALYZE` statement for a join between `orders` and `customers` tables, filtered by `customers.signup_date >= '2026-02-01'`.

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' section. The plan details the execution steps, including a Hash Join, Hash Cond, Seq Scan on `orders`, and Seq Scan on `customers`, along with their respective costs, widths, and execution times.

At the bottom of the interface, a status bar indicates 'Total rows: 14', 'Query complete 00:00:00.079', and the current cursor position 'Ln 2, Col 77'.

```

1  EXPLAIN ANALYZE
2  SELECT orders.id,customers.name,orders.total,orders.status,orders.created_at
3  FROM orders
4  INNER JOIN customers
5  ON orders.customer_id = customers.id
6  WHERE customers.signup_date >= '2026-02-01';

```

Showing rows: 1 to 14 | Page No: 1 of 1

QUERY PLAN

text

1	Hash Join (cost=12.84..14.47 rows=17 width=302) (actual time=0.037..0.046 rows=35.00 loops=1)
2	Hash Cond: (orders.customer_id = customers.id)
3	Buffers: shared hit=2
4	-> Seq Scan on orders (cost=0.00..1.50 rows=50 width=88) (actual time=0.016..0.017 rows=50.00 loops=1)
5	Buffers: shared hit=1
6	-> Hash (cost=12.12..12.12 rows=57 width=222) (actual time=0.014..0.014 rows=14.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on customers (cost=0.00..12.12 rows=57 width=222) (actual time=0.008..0.010 rows=14.00 loops=1)
10	Filter: (signup_date >= '2026-02-01':date)
11	Rows Removed by Filter: 6
12	Buffers: shared hit=1
13	Planning Time: 0.161 ms
14	Execution Time: 0.073 ms

Total rows: 14 | Query complete 00:00:00.079 | CRLF | Ln 2, Col 77

Original Query (Subquery) → Execution Time = 0.160 ms

Rewritten Query (JOIN) → Execution Time = 0.073 ms

Final Challenge — A Real-World Problem

Concepts used: this task combines everything from Tasks 2, 3, and 4 — Joins, Indexing, and Query Optimization — in one realistic scenario.

TASK 5 The Customer Profile Page

Scenario: a food-delivery company shows each customer their 5 most recent orders on their profile page — the order total, status, and the name of the restaurant it came from. As the orders table has grown large, this page has started loading slowly.

Complete the following steps, in order:

- a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```

1 SELECT
2     restaurants.name AS restaurant_name,
3     orders.total,
4     orders.status,
5     orders.created_at
6 FROM orders
7 INNER JOIN restaurants
8 ON orders.restaurant_id = restaurants.id
9 WHERE orders.customer_id = 11
10 ORDER BY orders.created_at DESC
11 LIMIT 5;
  
```

The results pane shows 4 rows of data:

	restaurant_name character varying (150)	total numeric (8,2)	status character varying (20)	created_at timestamp without time zone
1	Nile Taste	700.00	Completed	2026-08-28 14:40:00
2	Carnival Kitchen	610.00	Completed	2026-07-24 10:35:00
3	Rio Flavors	380.00	Pending	2026-06-07 15:00:00
4	Amazon Spice	550.00	Completed	2026-06-03 13:25:00

- b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

The screenshot shows the SQL IDE with the query editor and the results pane. The query is the same as in the previous screenshot, but with `EXPLAIN ANALYZE` added at the beginning. The results pane shows the query plan and execution statistics.

```

1 EXPLAIN ANALYZE
2 SELECT
3     restaurants.name AS restaurant_name,
4     orders.total,
5     orders.status,
6     orders.created_at
7 FROM orders
8 INNER JOIN restaurants
9 ON orders.restaurant_id = restaurants.id
10 WHERE orders.customer_id = 11
11 ORDER BY orders.created_at DESC
12 LIMIT 5;
  
```

The query plan shows the following steps:

- Limit (cost=9.91..9.92 rows=1 width=396) (actual time=0.048..0.049 rows=4 loops=1)
- Buffers: shared hit=9
- Sort (cost=9.91..9.92 rows=1 width=396) (actual time=0.048..0.048 rows=4 loops=1)
- Sort Key: orders.created_at DESC
- Sort Method: quicksort Memory: 256kB
- Buffers: shared hit=9
- Nested Loop (cost=0.14..9.90 rows=1 width=396) (actual time=0.035..0.042 rows=4 loops=1)
- Buffers: shared hit=9
- Seq Scan on orders (cost=0.00..1.62 rows=1 width=84) (actual time=0.017..0.019 rows=4 loops=1)
- Filter: (customer_id = 11)

The screenshot shows the execution statistics for the query. The results pane shows the following information:

- Rows Removed by Filter: 46
- Buffers: shared hit=1
- Index Scan using restaurants_pkey on restaurants (cost=0.14..8.16 rows=1 width=322) (actual time=0.005..0.005 rows=1 loops=1)
- Index Cond: (id = orders.restaurant_id)
- Index Searches: 4
- Buffers: shared hit=8
- Planning Time: 0.141 ms
- Execution Time: 0.069 ms
- Total rows: 18 Query complete 00:00:00.088

- c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

Query Query History

Show queries generated internally by pgAdmin? ☒ Remove Remove All

7/14/2026 7:05:23 PM 190 msec

Date Rows affected Duration

Copy Copy to Query Editor

```
CREATE INDEX idx_orders_status ON orders(status);
```

Messages

Query returned successfully in 190 msec.

- d) Re-run the same query and compare the new numbers to step (b).

Query Query History

```
1 EXPLAIN ANALYZE
2 SELECT
3     restaurants.name AS restaurant_name,
4     orders.total,
5     orders.status,
6     orders.created_at
7 FROM orders
8 INNER JOIN restaurants
9 ON orders.restaurant_id = restaurants.id
10 WHERE orders.customer_id = 11
11 ORDER BY orders.created_at DESC
12 LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1 of 1

QUERY PLAN

Limit (cost=9.91..9.92 rows=1 width=398) (actual time=0.054..0.055 rows=4.00 loops=1)

Buffers: shared hit=9

-> Sort (cost=9.91..9.92 rows=1 width=398) (actual time=0.053..0.054 rows=4.00 loops=1)

Sort Key: orders.created_at DESC

Sort Method: quicksort Memory: 25kB

Buffers: shared hit=9

-> Nested Loop (cost=0.14..9.90 rows=1 width=398) (actual time=0.032..0.043 rows=4.00 loops=1)

Buffers: shared hit=9

-> Seq Scan on orders (cost=0.00..1.62 rows=1 width=84) (actual time=0.022..0.025 rows=4.00 loops=1)

Filter: (customer_id = 11)

Rows Removed by Filter: 46

Buffers: shared hit=1

-> Index Scan using restaurants_pkey on restaurants (cost=0.14..8.16 rows=1 width=322) (actual time=0.003..0.003 rows=1.00 loops=1)

Index Cond: (id = orders.restaurant_id)

Index Searches: 4

Buffers: shared hit=8

Planning:

Buffers: shared hit=22 read=1

Planning Time: 1.834 ms

Execution Time: 0.087 ms

Total rows: 20 Query complete 00:00:00.092 CRLF Ln 12, Col 9

- e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

Query Query History

```
1 SELECT
2     restaurants.name AS restaurant_name,
3     orders.total,
4     orders.status,
5     orders.created_at
6 FROM orders
7 JOIN restaurants
8 ON orders.restaurant_id = restaurants.id
9 WHERE orders.customer_id = 11
10 ORDER BY orders.created_at DESC
11 LIMIT 5;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

restaurant_name	total	status	created_at
Nile Taste	700.00	Completed	2026-08-28 14:40:00
Carnival Kitchen	610.00	Completed	2026-07-24 10:35:00
Rio Flavors	380.00	Pending	2026-06-07 15:00:00
Amazon Spice	550.00	Completed	2026-06-03 13:25:00

- f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.
 - ⇒ Before creating the index, PostgreSQL had to scan more rows to find the required customer orders. After creating the composite index on `customer_id` and `created_at`, the query became more efficient because the database could quickly locate and sort the matching records. Since this practice database contains only 50 orders, the improvement is small, but with thousands of records the performance gain would be much greater.

Submission Checklist

- ☐ Your CREATE TABLE and INSERT statements from Part 1
- ☐ Screenshots of every query and its result for Tasks 1–4
- ☐ Written answers where requested (Task 1c, Task 2c)
- ☐ Full before/after comparison and explanation for the Final Challenge
- ☐ Submit everything as a single PDF or document, with each screenshot clearly labeled by task number

Reminder: if a query doesn't behave the way you expect, that's useful too — screenshot it and write one line on what you think went wrong. Understanding a mistake is worth as much as a clean result.

Issue Encountered: Initially, the orders table was created successfully, but no records were inserted because the INSERT INTO orders statement was not executed correctly. As a result, SELECT COUNT(*) FROM orders; returned 0. After identifying the issue, the complete INSERT INTO orders query was executed again, and the table was successfully populated with **50 records**.

